

RecruitmentAssist Backend and LangGraph

Simple explanation of the backend folder and agent architecture

Tiny Meaning

The backend is the hidden kitchen of the HR app. The frontend is what users see. The backend stores data, reads resumes, checks job descriptions, matches candidates, schedules interviews, and talks to AI.

Main Idea

- routes are the doors where frontend requests enter.
- services are the workers that do business logic.
- database.py is the big notebook that remembers jobs, candidates, interviews, users, and audit logs.
- agents are AI helpers for reading, matching, ranking, summarizing, and chatting.
- LangGraph is the traffic controller that decides which agent runs first and next.

Like You Are 5

Imagine an office helper. One helper opens the door, one writes in the notebook, one reads papers, one compares papers, one makes a list of best people, and one explains the answer. LangGraph tells these helpers whose turn it is.

Backend Folder Structure

What each major folder/file does

app.py	Starts the Flask backend, connects routes, prepares database and frontend preference.
database.py	MongoDB helper. Creates, reads, updates, and deletes jobs, candidates, comparisons, interviews, users, runs, and audit logs.
routes/	API doors. Files like <code>jd_routes.py</code> , <code>candidate_routes.py</code> , <code>interview_routes.py</code> , and <code>agentic_routes.py</code> receive frontend requests.
services/	Business workers. They prepare JD data, candidate profiles, matching results, interview emails, dashboard data, and auth state.
agents/	AI helpers. JD agent, resume agent, matching agent, ranking agent, summary agent, recruiter chat agent, and router agent.
tools/	Small controlled tools used by agents, with a gateway and policy checks before important actions.
workflows/	Recipes for bigger tasks such as screening, recruiter chat, and app actions.
orchestration/	LangGraph and orchestrator layer. This is where the graph is built and executed.
app/	Text extraction, regex extraction, LLM extraction, matching rules, summaries, and utility functions.
security/	Safety guard. Checks prompt injection and controls what tools/actions are allowed.
schemas/	Request validation forms, especially for agentic payloads.
static/uploads/	Uploaded resumes and job description files.
tests/	Security tests that check validation, policy, prompt injection, and audit behavior.

Important Files and Functions

Simple jobs of the main Python parts

database.py

- create_job adds a job.
- create_candidate adds a candidate.
- upsert_comparison saves matching results.
- create_interview stores interview details.
- authenticate_user checks login.

jd_service.py

- allowed_file checks uploaded file types.
- create_job_from_upload reads and saves a JD.
- jd_details_payload prepares JD details for frontend.

candidate_service.py

- normalize_candidate_record cleans candidate data.
- candidates_payload builds candidate list.
- candidate_profile_payload builds one full candidate profile.

matching_service.py

- run_matching compares resumes with a JD.
- run_matching_from_paths compares already saved resume files.

interview_service.py

- generate_interview_email writes invite email.
- assert_slot_available checks interview time.
- send_email sends the message.

security files

- context_safety.py detects unsafe prompt text.
- policy.py decides which tool actions are allowed.

How LangGraph Works Here

The traffic controller for agents

Where It Lives

LangGraph is mainly used in RecruitmentAssist/backend/orchestration/hr_graph.py.

What It Does

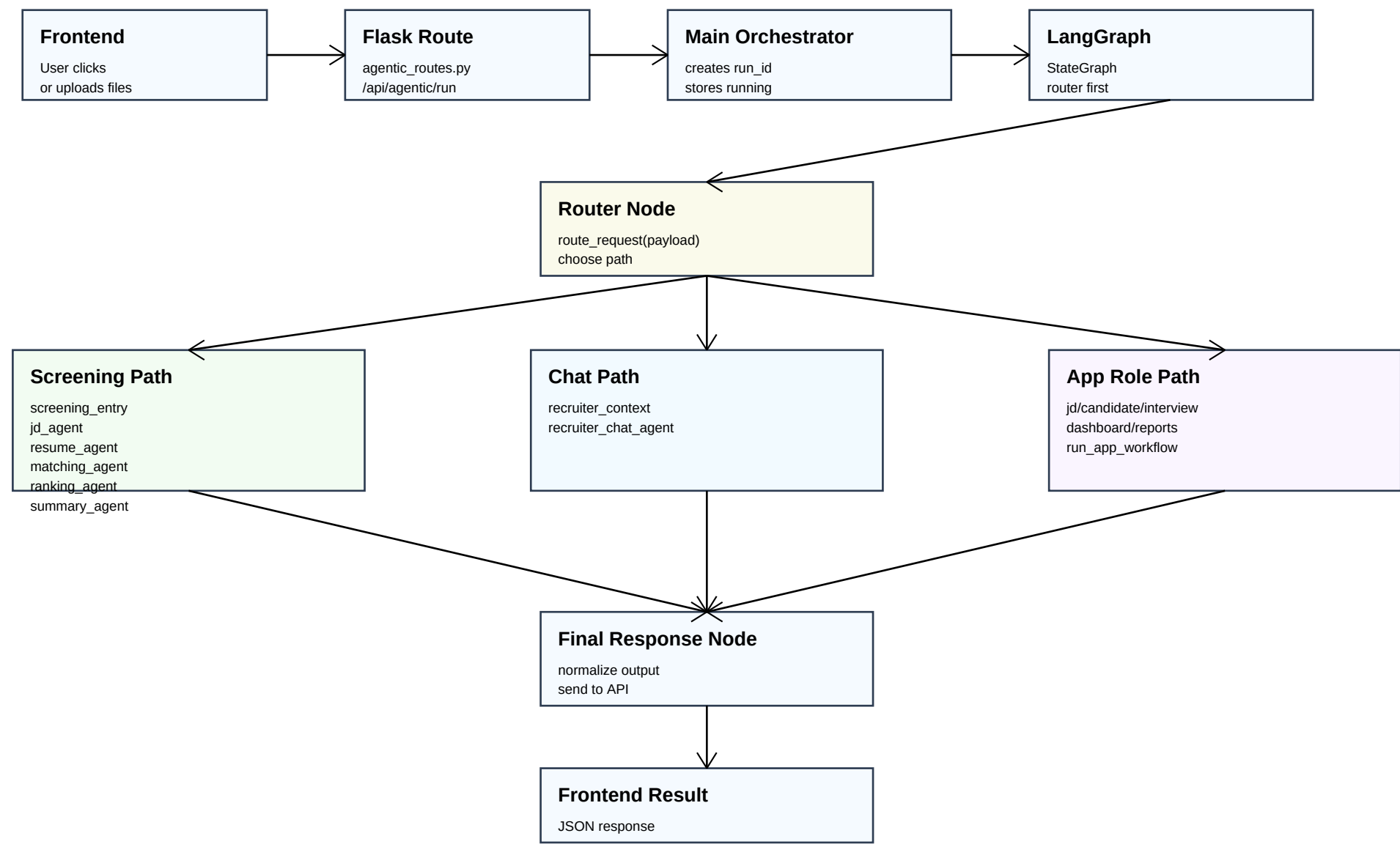
- It creates a StateGraph named HRGraphState.
- It adds nodes such as router, jd_agent, resume_agent, matching_agent, ranking_agent, summary_agent, recruiter_chat_agent, and final_response.
- It connects nodes with edges, which means after this step, go to that step.
- It uses conditional edges after the router so only the correct path runs.
- It returns one final response shape back to Flask and then the frontend.

The Shared Backpack

HRGraphState is like a backpack passed from node to node. It can carry payload, username, run_id, task_type, route decision, jd_id, JD context, resume context, match result, ranking result, summary, errors, and final_response.

LangGraph Flow Diagram

How a request travels through the graph



LangGraph Paths

What happens for each kind of request

Screening Request

Used when the app needs to screen candidates for a job description.

- screening_entry_node checks that jd_id exists.
- jd_agent_node loads and understands the selected job description.
- resume_agent_node turns candidate ids or uploaded resumes into clean profiles.
- matching_agent_node compares each profile with the JD and saves comparison records.
- ranking_agent_node sorts candidates into ranked/selected/rejected style output.
- summary_agent_node creates the final explanation.

Chat Request

- recruiter_context_node checks the message/action is valid.
- recruiter_chat_agent_node runs recruiter_chat_workflow.
- It can answer recruiter questions or prepare confirmed actions.

App Requests

- JD, candidate, interview, dashboard, and reports routes go through app_role_node.
- app_role_node calls run_app_workflow, which dispatches the right backend service/tool.

Request Lifecycle

From frontend request to final answer

1. Frontend sends JSON or uploaded files to `/api/agentic/run`.
2. `agentic_routes.py` normalizes the request into one payload.
3. `validate_agentic_payload` checks that the request is allowed and shaped correctly.
4. `run_main_orchestrator` creates a `run_id` and saves an agent run as running.
5. `run_hr_graph` invokes the compiled LangGraph graph.
6. `router_node` chooses the path.
7. The selected nodes run and update `HRGraphState`.
8. `final_response_node` creates the final API response.
9. `main_orchestrator` saves completed output, or failed error if something breaks.
10. Flask sends JSON back to the frontend.

Why This Is Useful

- The app does not need one huge messy function.
- Each agent has one clear job.
- The graph makes the order easy to see.
- Failed runs can be saved and inspected.
- New paths can be added later by adding nodes and edges.